Assignment 2: HTTP APIs, gRPC, and Metrics Report

Muhammad Abdullah Asim, 10020

For this assignment I worked on the swipe feature of WatchIt (`POST /api/swipes`). I picked it as my one user task because it saves data in the database, it is safe to repeat, it doesn't use any AI credits, and it is small enough to rebuild as a separate gRPC service.

Part A: Improve the HTTP API

1. REST principles

REST rule	Required?	What it means	Example from WatchIt
Client-server	Required	The frontend and the backend are separate programs.	The React app shows things and sends requests. The Express server owns the database.
Stateless	Required	The server doesn't remember anything between requests. Each request brings what the server needs.	The app sends its login token (`Authorization: Bearer ...`) on every request. There are no server sessions.
Cache	Required	Responses say whether they may be saved and reused.	Private data like `/api/profile` and `/api/swipes` says `Cache-Control: no-store`. `/api/movies` may be cached privately for a short time.
Uniform interface	Required	Every endpoint follows the same style: URLs name things, standard methods (GET, POST...), JSON bodies, and links to the next steps (called HATEOAS).	Partly. WatchIt uses clean URLs like `/api/swipes`, standard methods and JSON. It does not send links to next steps, so HATEOAS is not demonstrated.
Layered system	Required	The client doesn't need to know what sits between it and the server.	The React app just calls `/api/...`. It doesn't know whether a proxy or Vercel is in between.
Code on demand	**Optional**	The server can send code for the client to run.	Not used. The API only returns JSON.

2. OpenAPI

OpenAPI is a standard file format for describing an HTTP API: its routes, what each one accepts, and what it returns, including errors. Because the file can be read by tools, it can be used to show documentation, test endpoints, and check that the API and its description agree. I wrote an `openapi.yaml` file for WatchIt, served it with Swagger UI at `/api/docs`, and used Swagger UI to test `POST /api/swipes` (screenshots below).

3. What I changed in the swipe API

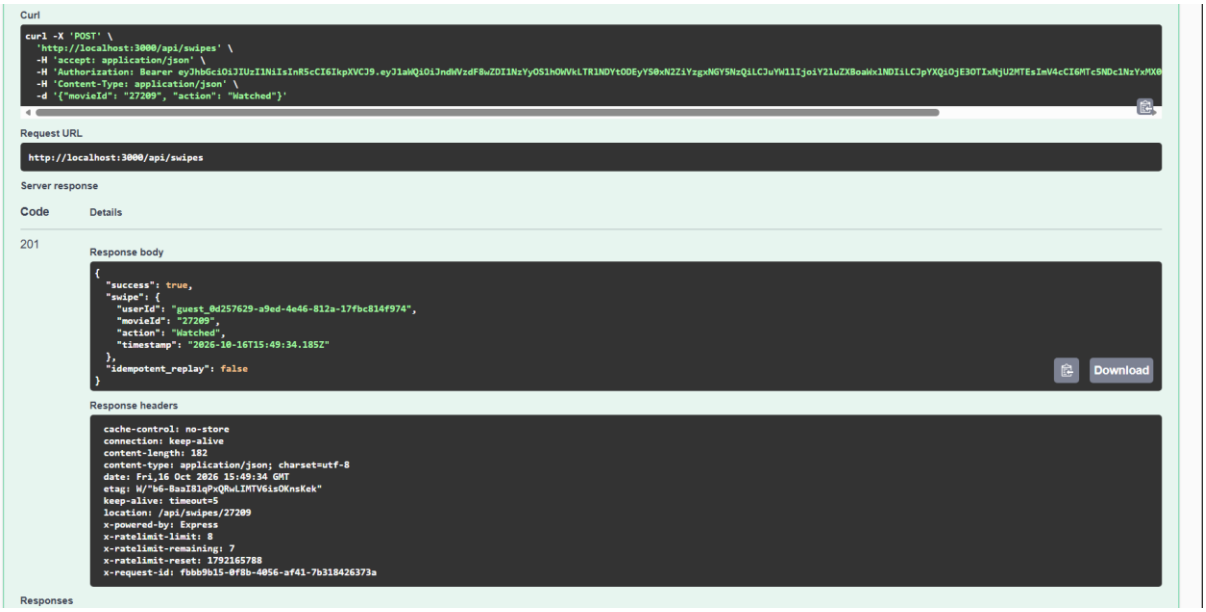
Problem	Before	After	Why
Name	`POST /api/swipe`	`POST /api/swipes` (the old name still works)	Collections are named with plural nouns. Keeping the old name means old callers don't break.
Status codes	Always `200 OK`	`201 Created` for a new swipe, `200 OK` if it already existed. A `Location` header points to the swipe.	The status tells the caller what really happened.
No input checks	Missing or wrong data crashed the server	Checks that `movieId` is there and `action` is one of the allowed words. Bad input gets `400 Bad Request`.	Input from callers can't be trusted, so check it before touching the database.
Messy errors	`{ "error": "Internal server error" }`	Every error uses one standard format (called Problem Details, RFC 9457) with `type`, `title`, `status`, `detail`, `instance` and `request_id`	Callers can read every error the same way. I kept the old `error` field too, so the React app still works.

Other endpoints. I reviewed every other endpoint (not counting health and metrics). All of them now return errors in the same standard format and check their input before using the database.

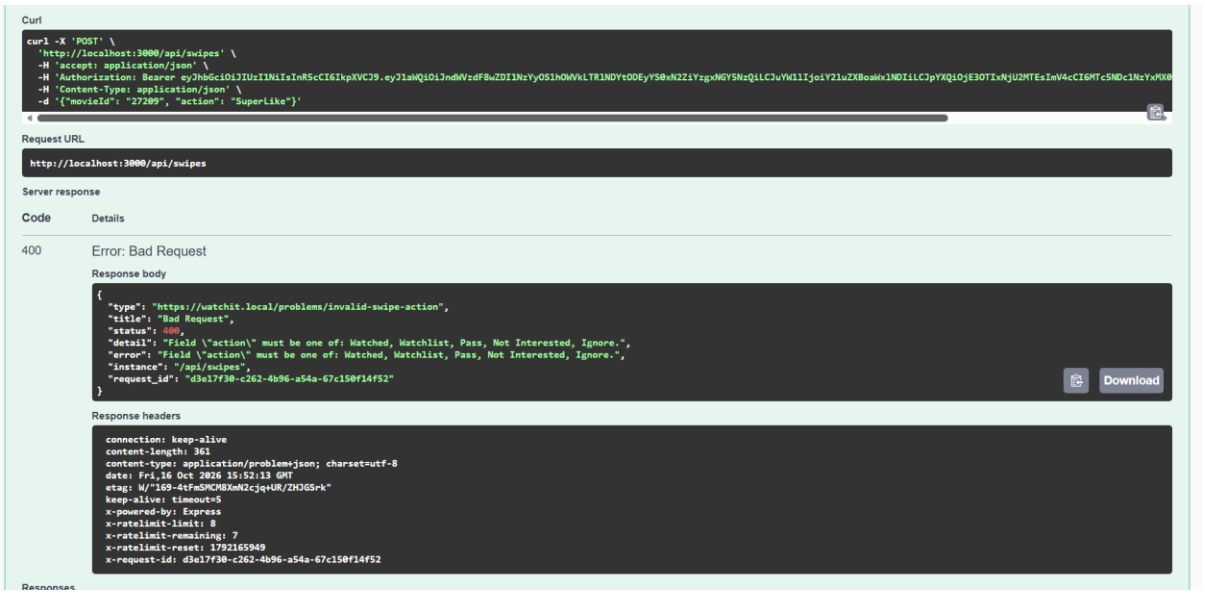
Endpoint	What it does	Checks and errors
`POST /api/auth/register`	Makes a guest account and returns a token	`400` bad input, `409` name taken
`POST /api/auth/login`	Logs in and returns a token	`400` bad input, `401` wrong details
`GET /api/movies`	Lists movies you haven't swiped	`400` if `page` is not a whole number from 1 to 500
`GET /api/swipes`	Lists your swipes (optional `?action=` filter)	`400` for an unknown action
`POST /api/swipes`	**Saves a swipe (my main task)**	`201`, `200`, `400`, `401`, `429`
`PUT` / `PATCH /api/swipes/{movieId}`	Changes an existing swipe	`400` bad action, `404` not found
`DELETE /api/swipes/{movieId}`	Removes a swipe	`400` if there is no movie id
`GET /api/profile`	Shows your profile and history	`404` if the user is missing
`PUT` / `PATCH /api/profile`	Changes name, bio and avatar	`400` if the name is empty or over 60 characters, or the bio is over 300
`GET /api/conversations` and `/{id}`	Lists chats, or shows one chat	`400` no id, `404` not found
`POST /api/chat`	Asks Cine Noir for recommendations	`400` if the question is empty or over 500 characters

4. Testing the swipe endpoint (Swagger UI)

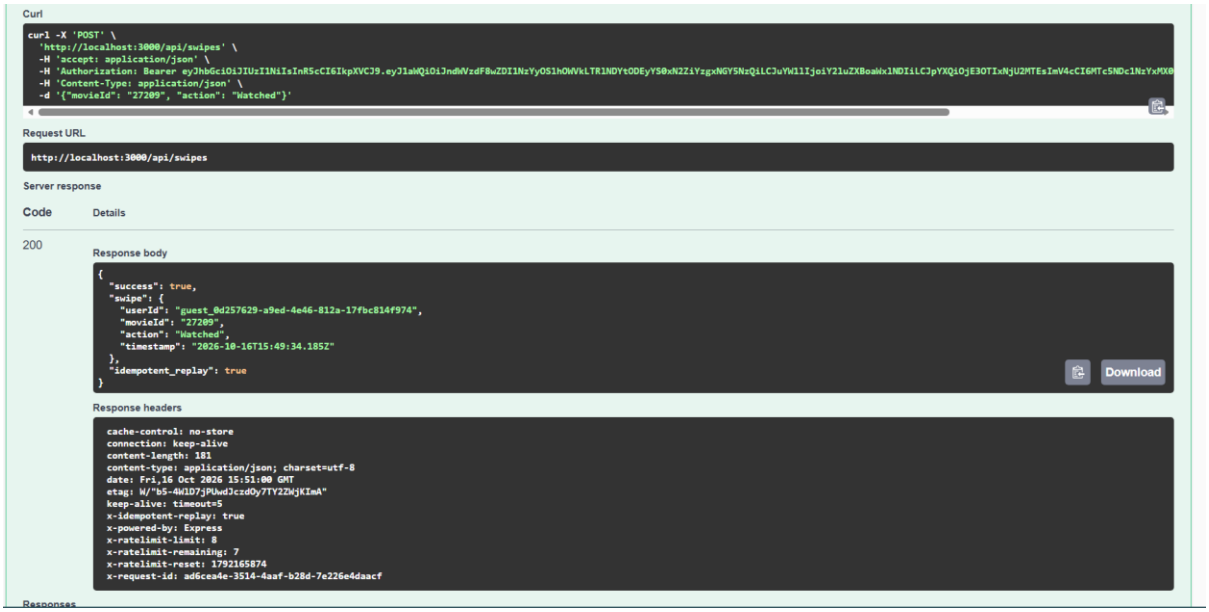
Valid request: `201 Created`



Invalid input: `400 Bad Request`



Repeated request: `200 OK` with `idempotent_replay: true`



5. Safe repeats

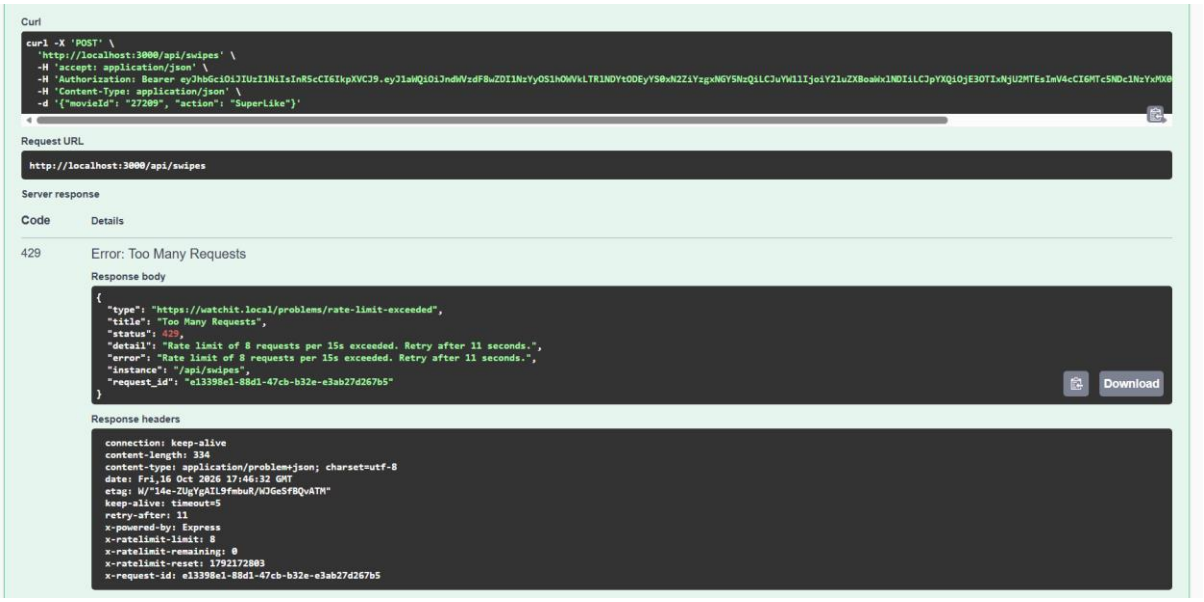
If the same swipe is sent twice (for example because the network glitched), the second one must not do anything extra. Two things protect `POST /api/swipes`:

- **Idempotency-Key:** the caller can send an `Idempotency-Key` header. If the server sees the same key from the same user again within 10 minutes, it sends back the same answer instead of doing the work again.
- **The database save:** even without a key, the swipe is saved as "insert or update" for that user and movie, and the server checks whether the same action is already saved. A repeat can't create a second row.

A repeat gets `200` with `"idempotent_replay": true`, and it isn't counted again in the swipe metric.

6. Rate limiting

- **Limit:** 8 requests per 15 seconds on `POST /api/swipes`.
- **Who shares it:** requests with the same `X-Client-Id` header. If a request has no such header, it is the same logged-in user.
- **Too many requests:** the server answers `429 Too Many Requests` with a `Retry-After` header (the seconds left to wait).
- **Allowed again:** when the 15-second window ends. Then the caller gets 8 new requests.
- **Note:** the counters are kept in the server's memory, which is fine for one server. They reset if the server restarts.



In this Swagger UI call, a request over the limit gets `429` with `Retry-After: 11` and `X-RateLimit-Remaining: 0`.

7. Python caller (http_client.py)

This script acts like another service calling `POST /api/swipes`. Run it with `python http_client.py`.

Settings

- **Timeout:** 1.5 seconds per try.
- **Retries:** up to 2 retries (3 tries in total). It waits 0.5 seconds, then 1.0 second.
- **What is retried:** only timeouts and `5xx` errors. A `4xx` answer is a real answer, so it is not retried. Retrying is safe because every try uses the same `Idempotency-Key`.
- **Circuit breaker:** opens after 3 failed calls in a row. While open, calls fail straight away. After 3 seconds it lets one call through, and if that works, the breaker closes again.

How I made failures happen on purpose. The script sends a test-only header, `X-Simulate-Fault`. With `503` the server returns a `503` straight away. With `timeout` the server waits 2.5 seconds, which is longer than the 1.5 second timeout.

What the script does

1. Normal calls: a valid swipe, the same swipe again (a safe repeat), and an invalid action.
2. A `503` on the first try, then a retry that works.
3. Three failing calls in a row (one with timeouts, two with `503`), which opens the breaker.

4. Two calls blocked while the breaker is open, a wait, then a call that works and closes the breaker.

5. A separate rate limit test: 10 swipes in a row.

Output

```
    ======================================================================
    WATCHIT HTTP RESILIENCE & RATE-LIMIT CALLER (PART A)
    Settings: Timeout=1.5s | MaxRetries=2 | Backoff=0.5s*2^n
    CircuitBreaker: FailureThreshold=3 | RecoveryPeriod=3.0s
    ======================================================================


    [SCENARIO 1] Valid Request, Idempotent Repeat, and Invalid Input
      -> Attempt 1/3 (Idempotency-Key=idem-05c..., fault=none)
      1a) Valid New Swipe     -> HTTP 201 | replay=False | action=Watched
      -> Attempt 1/3 (Idempotency-Key=idem-05c..., fault=none)
      1b) Repeated Request    -> HTTP 200 | replay=True (No duplicate effect!)
      -> Attempt 1/3 (Idempotency-Key=idem-61f..., fault=none)
      1c) Invalid Input Test  -> HTTP 400 | title=Bad Request | detail=Field "action"
must be one of: Watched, Watchlist, Pass, Not Interested, Ignore.


    [SCENARIO 2] Transient Failure (503 on Attempt 1 -> Retry with Backoff -> 200/201 on
Attempt 2)
      -> Attempt 1/3 (Idempotency-Key=idem-35e..., fault=503)
         [!] Attempt 1 failed: HTTP 503: Simulated transient upstream failure.
         [~] Retrying in 0.5s (safe due to Idempotency-Key)...
      -> Attempt 2/3 (Idempotency-Key=idem-35e..., fault=none)
      Result after retry -> HTTP 200 | success=True


    [SCENARIO 3] Repeated Failures -> Circuit Breaker Trips OPEN


      --- Failing Call #1 ---
      -> Attempt 1/3 (Idempotency-Key=idem-fai..., fault=timeout)
         [!] Attempt 1 failed: Timeout (> 1.5s)
         [~] Retrying in 0.5s (safe due to Idempotency-Key)...
      -> Attempt 2/3 (Idempotency-Key=idem-fai..., fault=timeout)
         [!] Attempt 2 failed: Timeout (> 1.5s)
         [~] Retrying in 1.0s (safe due to Idempotency-Key)...
      -> Attempt 3/3 (Idempotency-Key=idem-fai..., fault=timeout)
         [!] Attempt 3 failed: Timeout (> 1.5s)
         [X] Exhausted all 3 attempts.
      [CircuitBreaker] Recorded failure #1/3
      Call #1 final outcome: timed out


      --- Failing Call #2 ---
      -> Attempt 1/3 (Idempotency-Key=idem-fai..., fault=503)
         [!] Attempt 1 failed: HTTP 503: Simulated transient upstream failure.
         [~] Retrying in 0.5s (safe due to Idempotency-Key)...
      -> Attempt 2/3 (Idempotency-Key=idem-fai..., fault=503)
         [!] Attempt 2 failed: HTTP 503: Simulated transient upstream failure.
         [~] Retrying in 1.0s (safe due to Idempotency-Key)...
      -> Attempt 3/3 (Idempotency-Key=idem-fai..., fault=503)
         [!] Attempt 3 failed: HTTP 503: Simulated transient upstream failure.
         [X] Exhausted all 3 attempts.
      [CircuitBreaker] Recorded failure #2/3
      Call #2 final outcome: HTTP 503: Simulated transient upstream failure.


      --- Failing Call #3 ---
      -> Attempt 1/3 (Idempotency-Key=idem-fai..., fault=503)
         [!] Attempt 1 failed: HTTP 503: Simulated transient upstream failure.
         [~] Retrying in 0.5s (safe due to Idempotency-Key)...
      -> Attempt 2/3 (Idempotency-Key=idem-fai..., fault=503)
         [!] Attempt 2 failed: HTTP 503: Simulated transient upstream failure.
         [~] Retrying in 1.0s (safe due to Idempotency-Key)...
      -> Attempt 3/3 (Idempotency-Key=idem-fai..., fault=503)
         [!] Attempt 3 failed: HTTP 503: Simulated transient upstream failure.
         [X] Exhausted all 3 attempts.
```

```
      [CircuitBreaker] Recorded failure #3/3
      [CircuitBreaker] Threshold reached! State -> OPEN for 3.0s
      Call #3 final outcome: HTTP 503: Simulated transient upstream failure.


    [SCENARIO 4] Calls Blocked While Breaker is OPEN -> Cooldown -> Recovery
      Attempting call while breaker is OPEN...
      [BLOCKED FAST] Circuit Breaker is OPEN (fast-failing call; 3.0s remaining in
cooldown)
      Attempting call while breaker is OPEN...
      [BLOCKED FAST] Circuit Breaker is OPEN (fast-failing call; 3.0s remaining in
cooldown)


      Waiting 3.2s for Circuit Breaker recovery period...
      [CircuitBreaker] Cooldown (3.0s) elapsed -> State: HALF OPEN (probing server)
      -> Attempt 1/3 (Idempotency-Key=idem-rec..., fault=none)
         [!] Attempt 1 failed: Timeout (> 1.5s)
         [~] Retrying in 0.5s (safe due to Idempotency-Key)...
      -> Attempt 2/3 (Idempotency-Key=idem-rec..., fault=none)
      [CircuitBreaker] Call succeeded -> Resetting failure count to 0, State: CLOSED
      Recovered Call Outcome -> HTTP 200 | Breaker State: CLOSED


    [SCENARIO 5] Separate Rate Limiting Demonstration (Limit: 8 requests per 15s window)
      Request #01 -> HTTP 201 ACCEPTED          | Remaining: 7
      Request #02 -> HTTP 201 ACCEPTED          | Remaining: 6
      Request #03 -> HTTP 201 ACCEPTED          | Remaining: 5
      Request #04 -> HTTP 201 ACCEPTED          | Remaining: 4
      Request #05 -> HTTP 201 ACCEPTED          | Remaining: 3
      Request #06 -> HTTP 201 ACCEPTED          | Remaining: 2
      Request #07 -> HTTP 201 ACCEPTED          | Remaining: 1
      Request #08 -> HTTP 201 ACCEPTED          | Remaining: 7
      Request #09 -> HTTP 201 ACCEPTED          | Remaining: 6
      Request #10 -> HTTP 201 ACCEPTED          | Remaining: 5
```

What the output shows

- Scenarios 1 to 4 show a success, a safe repeat, a retry after a `503`, the breaker opening, calls blocked while it is open, and recovery. In scenario 4 the first try of the recovery call hit a real timeout, so it was retried and the second try worked.
- Scenario 5 never reached a `429`. `Remaining` counts down from 7 to 1, then jumps back to 7 at request #08. The first 7 requests took more than 15 seconds in total, so the window ended and a new one started. That is the window resetting, but it means this loop never went over the limit, so I showed the `429` in Swagger UI instead (section 6).

Part B: gRPC service

I rebuilt the swipe save as its own gRPC service (`grpc-server.ts`). It runs as a separate process on port 50051. It has its own checks and its own database code (it is not a wrapper around the HTTP route), and it saves to the same PostgreSQL database, so both versions share one source of truth. Both accept the same five actions: Watched, Watchlist, Pass, Not Interested and Ignore.

1. file and generated code `.proto` file and generated code

A `.proto` file describes the call: its name, what goes in, and what comes out. Mine is `proto/swipes.proto`. It has one call, `RecordSwipe`: one request in, one reply out.

- **In (`SwipeRequest`):** `movie_id`, `action`, `request_id`, `user_id`
- **Out (`SwipeResponse`):** `success`, `movie_id`, `action`, `idempotent_replay`

I generated the Python client code from the `.proto` file with this command:

```
    python -m grpc_tools.protoc -I proto --python_out=. --
grpc_python_out=. proto/swipes.proto
```

This creates `swipes_pb2.py` (the message classes) and `swipes_pb2_grpc.py` (the client stub). The Node server reads the `.proto` file when it starts, so it doesn't need generated files.

One shortcut: a real system would get `user_id` from the login. Here the client sends it in the request to keep things simple.

2. Python caller (grpc_client.py)

It calls the gRPC service directly with a 5 second deadline and prints either the result or the error. Run it with `python grpc_client.py`. It loops 20 times, and each loop sends one valid swipe and one invalid swipe (action `SuperLike`).

3. Call paths

gRPC

```
Python client (grpc_client.py)
   |  calls stub.RecordSwipe(...) with a 5 second deadline
   ▼
Generated client stub (swipes_pb2_grpc.py)
   • packs the request into a small binary message
   • adds the request ID
   • sends it over the network (HTTP/2)
   |
   ▼
gRPC server (grpc-server.ts, port 50051)
   • receives the message and unpacks it
   |
   ▼
Handler (recordSwipe)
   1. logs the request ID
   2. checks movie_id and action (INVALID_ARGUMENT if bad)
   3. saves the swipe in PostgreSQL (insert or update)
```

```
       4. updates the metrics and sends the reply
```

HTTP. There is no generated stub here. The caller builds the JSON by hand.

```
    Python caller (http_client.py)
       • builds the JSON by hand
       • adds headers: login token, Idempotency-Key, request ID
       |  POST /api/swipes (1.5 second timeout)
       ▼
    Express server (app.ts, port 3000)
       1. gives the request an ID and logs it
       2. starts the metrics timer (so even rejected calls are counted)
       3. checks the login token (401 if bad)
       4. rate limiter (429 if over 8 per 15 seconds)
       |
       ▼
    Swipe handler (POST /api/swipes)
       5. checks movieId and action (400 if bad)
       6. looks for a repeat (Idempotency-Key, or the same swipe
already saved)
       7. saves the swipe in PostgreSQL (insert or update)
       8. updates the metrics and replies 201 or 200
```

4. One success and one failure, both versions

HTTP (from `python http_client.py`)

```
    1a) Valid New Swipe     -> HTTP 201 | replay=False | action=Watched
    1c) Invalid Input Test  -> HTTP 400 | title=Bad Request | detail=Field
"action" must be one of: Watched, Watchlist, Pass, Not Interested, Ignore.
```

gRPC (from `python grpc_client.py`, first loop of 20)

```
    --- Sending gRPC Request (Trace ID: trace-027d3cc4-348e-4197-b976-
1b2218224e63) ---
    Input Data: movieId='grpc-movie-1792238175-1', action='Watched'
    [SUCCESS] gRPC Call Completed!
      Movie ID: grpc-movie-1792238175-1
      Action: Watched
      Idempotent Replay: False

    --- Sending gRPC Request (Trace ID: trace-2f304566-d472-4bc5-9faf-
73f139adae17) ---
    Input Data: movieId='12345', action='SuperLike'
    [FAILED] gRPC Call Rejected!
      gRPC Status Code: StatusCode.INVALID_ARGUMENT
      Error Details: Field "action" must be one of: Watched, Watchlist, Pass,
```

```
Not Interested, Ignore
```

The other 19 loops look the same, so one run gives 20 successes and 20 failures.

What the statuses mean

Situation	HTTP	gRPC	Why they differ
Swipe saved	`201 Created` (new) or `200 OK` (already there)	`OK`	HTTP describes what happened to the thing on the server. gRPC works like calling a function: it either worked (`OK`) or it didn't.
Bad action word	`400 Bad Request`	`INVALID_ARGUMENT`	Both mean "your input is wrong".
Too slow	No status. The caller's 1.5 second timeout stops the wait.	`DEADLINE_EXCEEDED`	gRPC has deadlines built in, so the client's 5 second deadline is reported as a status.

5. Following one request in the logs

Each request gets a **request ID**, a unique code that is written into every log line for that request. Searching Kibana for that ID shows everything that happened to it. For gRPC, the Python client makes the ID and sends it with the call (the name of the field is `x-request-id`). The gRPC server writes it into its logs with the same logger as the HTTP server, and tags its lines `service_name: "watchit-grpc-server"`. So I can search one request ID in Kibana and follow it across both servers. I also changed the HTTP server so it reuses a request ID sent by the caller instead of always making a new one.

Part C: Metrics

1. Checking my A1 metrics

Metric	Is it useful?	What I did
`watchit_movie_swipes_total`	Yes, it shows how much people swipe. But it only counts swipes that actually change something.	Kept.
`watchit_ai_wins_total`	Yes, it shows which AI answered.	Kept. It also counts the offline fallback (`Local_Vault`).

Metric	Is it useful?	What I did
`watchit_active_ai_requests`	Yes, it shows how many AI requests run at once.	Kept.
`watchit_ai_generation_duration_seconds`	Yes, it shows how slow the AI gets.	Kept.
`watchit_db_query_duration_seconds`	Only partly. It times one query in `GET /api/profile` and nothing else.	Kept. The new duration metrics below cover the swipe route.
Missing: HTTP swipe route	Nothing recorded how long `POST /api/swipes` takes, or how many calls fail with `400`, `429` or `5xx`.	**Added** two metrics (D and E below).
Missing: gRPC service	The gRPC server had no metrics at all.	**Added** three metrics (A, B and C below).

Prometheus reads the gRPC server's own `/metrics` page on port 9091 every 5 seconds (job `watchit-grpc-server` in `prometheus.yml`).

2. The new metrics

All durations are measured **on the server**, not the client. All labels have only a few fixed values. I never use user, request or trace IDs as labels.

A. gRPC calls by status (`watchit_grpc_calls_total`)

- **What it shows:** how many gRPC calls there are, and how many fail, by status.
- **Type and unit:** counter, calls
- **Labels:** `method` (`RecordSwipe`), `status` (`OK`, `INVALID_ARGUMENT`, `INTERNAL`)
- **Where in the code:** `grpc-server.ts`, just before each reply is sent
- **Query:** `sum(rate(watchit_grpc_calls_total[1m])) by (status, method)`

B. gRPC duration (`watchit_grpc_duration_seconds`)

- **What it shows:** how long a call takes, including the database save.
- **Type and unit:** histogram, seconds
- **Labels:** `method`
- **Buckets:** 0.1, 0.5, 1, 2, 3, 5, 8, 10, 15 seconds

- **Where in the code:** `grpc-server.ts`, the timer starts at the top of the handler and stops just before each reply
- **Queries (1 minute window):**
  - p50: `histogram_quantile(0.50, sum(rate(watchit_grpc_duration_seconds_bucket[1m])) by (le, method))`
  - p95: `histogram_quantile(0.95, sum(rate(watchit_grpc_duration_seconds_bucket[1m])) by (le, method))`

C. gRPC calls in progress (`watchit_grpc_inflight_requests`)

- **Why this one:** I picked in-flight calls instead of message size. I already have a similar gauge for AI requests (`watchit_active_ai_requests`), so the two are easy to compare, and swipe messages are tiny, so their size would tell me very little.
- **Type and unit:** gauge, calls
- **Labels:** `method`
- **Where in the code:** `grpc-server.ts`, goes up when a call starts and down just before the reply
- **Query:** `sum(watchit_grpc_inflight_requests)`

D. HTTP swipe calls by status (`watchit_http_swipe_calls_total`)

- **What it shows:** how many calls `POST /api/swipes` gets, and how many fail, by status code. It is the HTTP version of metric A.
- **Type and unit:** counter, calls
- **Labels:** `status` (for example `200`, `201`, `400`, `401`, `429`, `503`)
- **Where in the code:** defined in `server/metrics.ts`. A small first handler on the route in `app.ts` counts the call when the response finishes. It runs before the login check and the rate limiter, so rejected calls are counted too.
- **Query:** `sum(rate(watchit_http_swipe_calls_total[1m])) by (status)`

E. HTTP swipe duration (`watchit_http_swipe_duration_seconds`)

- **What it shows:** how long the whole route takes. It is the HTTP version of metric B.
- **Type and unit:** histogram, seconds
- **Labels:** none
- **Buckets:** the same as metric B, so the two can be compared
- **Where in the code:** the same first handler as metric D
- **Queries (1 minute window):**

- p50: `histogram_quantile(0.50, sum(rate(watchit_http_swipe_duration_seconds_bucket[1m])) by (le))`
- p95: `histogram_quantile(0.95, sum(rate(watchit_http_swipe_duration_seconds_bucket[1m])) by (le))`

3. Dashboard

I filled the panels by adding fake traffic by running `python grpc_client.py` (20 successes and 20 failures each time) and `python http_client.py`. The top row is gRPC and the bottom row is HTTP.



- **Duration:** both p95 lines are around 1 second or a little below, and both p50 lines are much lower. The gRPC p50 is low because half of the gRPC calls are the invalid ones, which are rejected before the database is touched.
- **Calls:** the gRPC panel shows `OK` and `INVALID_ARGUMENT`. The HTTP panel shows `200`, `201`, `400`, `429` and `503`, because the Python caller causes those failures on purpose.

The in-flight panel (metric C) is the gauge on the right:

